Mex: Scheme Without the Brackets, for VM16

Specification, version 0.1 (draft)

November 5, 2026

Abstract

Mex is a small language with the data and functions of Scheme (pairs, symbols, first-class functions with closures, proper tail calls) and the syntax of C (infix operators, braces, statements). It has two data structures: the pair, from which lists and trees are built, and the hash table, which serves as dictionary, record and array. It is designed to compile itself on VM16, running on a real Z80 or 6502 with 64 KB of memory. This document defines its syntax (in BNF), its values and their meaning, their representation on VM16, and the plan for bootstrapping the compiler.

1 Goals

1. **Self-hosting on 8-bit hardware.** The compiler is written in Mex, and it compiles itself on VM16 within 64 KB.
2. **A small core.** Six types of value, two data structures, fifteen built-in functions. What a larger language provides as a type (strings, arrays, records) is made from these.
3. **Simple to compile.** A one-pass recursive-descent compiler, with no syntax tree, emits VM16 code directly.

Non-goals: speed, floating point, macros, `call/cc`, and integers wider than 15 bits.

Related languages. The idea is old. McCarthy meant Lisp programs to be written as M-expressions, with S-expressions only for data; Logo is Lisp without the brackets; Dylan gave a Scheme-like core an infix syntax; and JavaScript and Lua are close relatives in all but their data. Mex takes the smallest possible version of the idea.

2 A first example

```
// Evaluate arithmetic expressions over variables.

var ops = {                       // operator -> function
    "+": fun (a, b) { return a + b; },
    "-": fun (a, b) { return a - b; },
    "*": fun (a, b) { return a * b; },
    "/": fun (a, b) { return a / b; }
};

// An expression is a table used as a record.
fun num(n)        { return {kind: "num", value: n}; }
fun ref(x)        { return {kind: "var", name: x}; }
fun bin(op, l, r) { return {kind: "op", op: op, left: l, right: r}; }
```

```
fun eval(e, env) {              // the value of e, with variables from env
    if (e.kind == "num") { return e.value; }
    if (e.kind == "var") { return env[e.name] || error("unbound variable"); }
    return ops[e.op](eval(e.left, env), eval(e.right, env));
}

fun vars(e, seen) {             // add the variables of e to the table seen
    if (e.kind == "var") { seen[e.name] = true; }
    if (e.kind == "op") { vars(e.left, seen); vars(e.right, seen); }
    return seen;
}

fun each(l, f) {                // call f on every element of the list l
    if (l == nil) { return nil; }
    f(hd(l));
    return each(tl(l), f);      // a tail call: this is a loop
}

fun main() {
    var e = bin("*", bin("+", ref("x"), num(3)),     // (x + 3) * (y - x)
                     bin("-", ref("y"), ref("x")));
    print(eval(e, {x: 4, y: 10}));                   // 42
    putc('\n');
    each(keys(vars(e, {})), fun (v) { print(v); putc(' '); });   // x y
    putc('\n');
}
```

Tables play three parts here. Each node of the expression is a *record*, a table with symbol keys read as `e.kind` and `e.left`. The variables and their values are a *dictionary*, `{x: 4, y: 10}`, and so is `seen`, used as a set. And `ops` is a *dispatch table* from operator symbols to functions, so `ops[e.op](a, b)` calls the right one. `keys` lists the variables in the order they were first stored, so the program always prints `x y`. The list it returns is made of pairs, the other data structure, and `each` walks it with a tail call, which runs in constant space.

A program is a series of global variable and function definitions. Running it first defines every function, then initialises the global variables in order, and then calls `main()`.

3 Lexical structure

token	form
name	a letter or `_`, then letters, digits and `_`
integer	decimal digits, or `0x` and hexadecimal digits; at most `0x7FFF`, taken modulo 2^{15}
character	`'a'`: one character or escape between single quotes; its value is an integer
symbol	`"text"`: characters and escapes between double quotes; its value is the symbol with that name
escape	`\n \t \r \0 \\ \' \"` `\x`*hh*
keyword	`fun var if else while return break continue nil true false`
operator	`( ) { } [ ] , ; : . = == != < <= > >= + - * / % ! && ||`
comment	`//` to the end of the line

Spaces, tabs and newlines separate tokens and have no other meaning. Names are case-sensitive. There is no string type: `"expected ;"` is a symbol, like `"if"`, and is what an error message or a keyword table is made of.

program ::= { *global* }
global ::= *vardecl* | `"fun"` NAME *function*
function ::= `"("` [*params*] `")"` *block*
params ::= NAME { `","` NAME }
vardecl ::= `"var"` NAME [`"="` *expr*] `";"`

block ::= `"{"` { *stmt* } `"}"`
stmt ::= *vardecl* | *block* | *if*
| `"while"` `"("` *expr* `")"` *block*
| `"return"` [*expr*] `";"` | `"break"` `";"` | `"continue"` `";"`
| *expr* [`"="` *expr*] `";"`
if ::= `"if"` `"("` *expr* `")"` *block* [`"else"` (*block* | *if*)]

expr ::= *and* { `"||"` *and* }
and ::= *compare* { `"&&"` *compare* }
compare ::= *cons* [(`"=="` | `"!="` | `"<"` | `"<="` | `">"` | `">="`) *cons*]
cons ::= *sum* [`":"` *cons*]
sum ::= *product* { (`"+"` | `"-"`) *product* }
product ::= *unary* { (`"*"` | `"/"` | `"%"`) *unary* }
unary ::= (`"-"` | `"!"`) *unary* | *postfix*
postfix ::= *primary* { `"("` [*args*] `")"` | `"["` *expr* `"]"` | `"."` NAME }
args ::= *expr* { `","` *expr* }
primary ::= NAME | INTEGER | CHARACTER | SYMBOL | `"nil"` | `"true"` | `"false"`
| `"("` *expr* `")"` | `"["` [*args*] `"]"` | *table* | `"fun"` *function*
table ::= `"{"` [*entry* { `","` *entry* }] `"}"`
entry ::= (NAME | SYMBOL | INTEGER | CHARACTER) `":"` *expr*

Figure 1: The grammar of Mex.

4 Grammar

In the grammar, $\{ x \}$ means zero or more x, $[x]$ means an optional x, and quoted text is a token. Precedence is encoded in the grammar, loosest first. The binary operators are left-associative, except `":"`, which is right-associative (`1 : 2 : nil` is `1 : (2 : nil)`), and the comparisons, which do not associate. The whole grammar is in fig. 1.

Notes.

- Braces are required around every body, so there is no dangling `else`.
- The left side of `"="` must be a NAME, or a *postfix* ending in `"["..."]"` or `"."`NAME. Assignment is a statement, not an expression. A pair is changed with `sethd` and `settl`.
- A *block* as a statement and a *table* as an expression never meet: a statement that starts with `"{"` is a block.
- In an *entry*, the key is one token, so the `":"` after it is never the pair operator: in `{a: 1 : nil}` the value is the list `1 : nil`.
- The grammar is LL(1). Whether an expression statement is an assignment is only known at the `"="`, so the compiler emits nothing for the last name or indexing step of an expression until it sees the next token.
- `"fun"` at the start of a global is a definition; anywhere in an expression it makes an

anonymous function. A statement that starts with `"fun"` is an expression statement.

5 Values and meaning

Every value has one of six types:

type	values
nil	the single value `nil`, which is the empty list and the only false value. `false` is another name for it
integer	−16384 to 16383. Arithmetic wraps modulo 2^{15}. A character is its code
pair	a cell holding two values, its head and its tail
symbol	a name. Two symbols with the same name are the same symbol. `true` is the symbol `"true"`
table	a hash table: a mutable map from keys to values
function	a function defined with `fun`, with the values of the variables it captured, or a built-in function

Truth. `nil` is false; every other value is true, including 0. The comparisons and `!` return `true` or `nil`. `a && b` is `nil` if `a` is, and otherwise `b`; `a || b` is `a` unless it is `nil`, and otherwise `b`. Neither evaluates `b` unless it needs to, so `lookup(k, env) || error("unbound")` is a common idiom.

Operators.

- `+ - * / %` take integers; `/` and `%` truncate towards zero, as in C, and division by zero is an error.
- `h : t` makes a new pair with head `h` and tail `t`. `[a, b, c]` is short for `a : b : c : nil`.
- `< <= > >=` compare two integers, or two symbols by their names in byte order, which is enough to keep a binary search tree.
- `==` and `!=` take any two values. Integers are equal when their values are; any other value, a table included, is equal only to itself. Because symbols are interned, two symbols are equal exactly when their names are. Whether two lists have equal elements is a function of the library.

Any other combination of operand types is a run-time error.

Tables. `t[k]` is the value stored under the key `k` in the table `t`, or `nil`. `t[k] = v` stores it, replacing any value already there, and storing `nil` removes the key. Any value but `nil` can be a key, and keys are compared as by `==`: an integer by its value, a symbol by its name, a pair, function or table by its identity. `t.name` is short for `t["name"]`, so a table with symbol keys is a record. `{a: 1, "b c": 2, 7: 3}` makes a new table; a NAME before `":"` is a symbol key.

`keys(t)` is a new list of the keys of `t` in the order they were first stored, counting a key that was removed and stored again as new. The order is therefore the same on every implementation and every run, which the bootstrap test (section 10) depends on. Because the list is a copy, a loop over it may change the table.

Variables and scope. A `var` at the top level declares a global; one inside a function declares a local, visible from the end of its declaration to the end of the enclosing block, so `var x = x;` reads an outer `x`. Parameters are locals, in the same scope as the function's outermost block. A name that is not a local refers to a global, which may be declared later in the program. Every global that is used must be declared somewhere, no name may be declared twice in the same scope, and the names of the built-in functions cannot be declared at all.

Functions and closures. Functions are values. `fun (x) {...}` in an expression makes a new function, which may use the locals of the functions around it. It captures their *values* when it is made: a local that a function captures cannot be assigned anywhere, before or after the capture, so the copy and the original can never differ. State that several functions share lives in a pair, a table or a global:

```
fun counter() {                 // a function that counts its calls
    var n = 0 : nil;            // a pair as a mutable box
    return fun () { sethd(n, hd(n) + 1); return hd(n); };
}
```

Calling a value that is not a function, or with the wrong number of arguments, is an error. A function that ends without `return` returns `nil`. Arguments are evaluated left to right, and the function itself before them.

Tail calls. In `return f(...);` the call is a tail call: it reuses the frame of the function that makes it, so a recursion in tail position runs in constant space, as in Scheme. `each` in the first example is a loop.

Errors. A run-time error prints a message and stops the program with exit status 1. There is no way to catch an error.

6 Built-in functions

The built-ins are global functions with reserved names.

function	result
`hd(p)`, `tl(p)`	the head or the tail of the pair `p`
`sethd(p, v)`, `settl(p, v)`	stores `v` as the head or the tail of `p`
`intern(l)`	the symbol whose name is the list of character codes `l`
`name(s)`	the name of the symbol `s`, as a new list of character codes
`keys(t)`	the keys of the table `t`, as a new list, in the order they were first stored
`size(t)`	the number of keys in the table `t`
`type(v)`	`"nil"`, `"int"`, `"pair"`, `"sym"`, `"table"` or `"fun"`
`getc()`	the next byte of standard input, or -1 at end of input
`putc(c)`	writes the byte c mod 256
`putw(n)`	writes n mod 2^{15} as a little-endian word
`print(v)`	writes an integer in decimal, or a symbol's name
`error(s)`	prints the symbol `s` and stops with an error
`exit(n)`	stops with exit status `n`

Everything else (`reverse`, `length`, `append`, `map`, `equal`, printing a list) is written in Mex.

7 Pairs or tables

The two structures cover each other's weak points.

Tables are the dictionary (`env[x]`), the record (`{op: "+", left: a, right: b}`, read as `e.left`) and the array (`t[0]`, `t[1]`, ..., with `size(t)` as the length). Lookup takes constant time on average, and an entry costs a few words.

Pairs are the list, and lists are *persistent*: putting a pair in front of a list leaves the list unchanged. That makes them the natural environment for nested scopes. An association list, a list of `key : value` pairs, gains a binding in constant time, and the old environment is simply the old list:

```
fun assoc(k, a) {                // the pair (k : v) in a, or nil
    while (a != nil) {
        if (hd(hd(a)) == k) { return hd(a); }
        a = tl(a);
    }
    return nil;
}
```

A string is a list of character codes, built with `:` and turned into a symbol by `intern` when it has to be compared or kept.

The compiler uses both. Globals go in a table from symbol to slot. The locals of the function being compiled are an association list from symbol to frame offset: entering a block saves the list and leaving it restores it, so scopes cost nothing. The code of one function is a list of bytes, newest first, and a jump is patched by keeping the pair that holds its placeholder and calling `sethd` on it when the target is known.

8 Representation on VM16

This section is not part of the language; it is the plan for the first implementation.

Values are single 16-bit words, told apart by their low bits and their address:

word	value
$2n + 1$ (odd)	the integer n
0	`nil`
even, below `HEAP`	a symbol: the address of its entry in the symbol area
$4k$, from `HEAP` up	a pair: the address of its cell
$4k + 2$, from `HEAP` up	a function or a table: the address of its cell plus 2

Integer arithmetic on tagged words is cheap ($a + b$ is $a + b - 1$), as in the CAM and the graph reducer of this repository. The type of a value is found from the value alone, except that a function and a table are told apart by the first word of their cell.

Cells. The heap is an array of two-word cells, aligned on 4 bytes, with no header. A pair's cell holds its head and tail. A function's cell holds the address of its code and its environment: `nil`, or the list of the values it captured, in the order the compiler numbered them. A built-in is a function with `nil` environment whose code is a runtime routine. Each global function is one cell, made when the program starts. A table's cell holds the address of its block (below) and `nil`: a code address is below `HEAP` and a block is above it, which is how the two are told apart.

Tables are compact hash tables, as in CPython. A table's block holds a header (the address of its cell, the capacity n, the number of entries used), an array of n entries `(key, value)` in the order they were stored, an index of $2n$ words, each 0 or an entry number plus one, searched by linear probing, and a last word holding the size of the block. Iteration walks the entries, which is where the insertion order of `keys` comes from. Removing a key sets its entry's value to `nil` and leaves the key as a tombstone. When the entries are used up, the table moves to a new block

of twice the capacity, or the same capacity if many entries were removed, copying only the live entries. The hash of an integer is its value, that of a symbol is computed from its name when it is interned and kept in its entry, and that of a pair, function or table is its address, which is fixed because cells never move. A new table has capacity 4, which costs 40 bytes: 6 of header, 16 of entries, 16 of index and 2 of size.

Symbols are kept in a symbol area below `HEAP`, which grows as symbols are interned and is never collected. An entry holds a link for the hash chain, the hash, the length and the bytes of the name. The symbols the runtime itself needs (`"true"`, the names `type` returns) are interned first, in a fixed order, so the compiler knows their addresses.

The heap holds cells, which are allocated from its bottom up, and table blocks, which are allocated from its top down. Cells never move; blocks do.

Garbage collection runs when the two meet. It marks, sweeps the cells and compacts the blocks.

- *Mark.* The roots are the data stack and the global table. The mark bits are a bitmap outside the heap, one bit per cell. Marking a table marks its cell and then the keys and values in its block. The marker keeps the objects still to scan on a small stack of fixed size, outside the heap. If the stack overflows, the marker notes it and, once the stack is empty, rescans the heap for marked objects that refer to unmarked ones, so a deep list costs time, not space.
- *Sweep.* Every cell has the same size, so the sweep links the unmarked cells into a free list, and the cells never fragment.
- *Compact.* Each block starts with the address of its cell and ends with its size, so the blocks can be walked from the top down. A block whose cell is marked and still points to it is slid up against the last one kept, and its cell is updated. Every other block (a dead table's, or one its table has moved out of) is dropped.

A function's first word is a code address, which the collector skips.

Frames. Arguments are pushed on the data stack left to right, then the function, and `CALL` pushes the return address on the return stack. The callee saves the frame pointer, pushes `nil` for each local, and addresses parameters and locals as `[fp±k]`; a captured value i is element i of the environment list, which the function finds in its own cell. A tail call copies its arguments and function over the caller's, restores the caller's frame pointer and jumps, so the return stack does not grow.

Code. The compiler generates code for a stack machine on VM16's data stack, with integer operations inline after a test of the low bits, and calls runtime routines for allocation, calls through values and comparisons of symbols.

9 Programs and the compiler

The object file. As for the graph reducer, the runtime `rt.v16` is the image: it reads the compiled program from the start of standard input, places it in memory and runs it, and the rest of standard input is the program's own:

```
cat prog.mxo input.txt | vm16 rt.v16
```

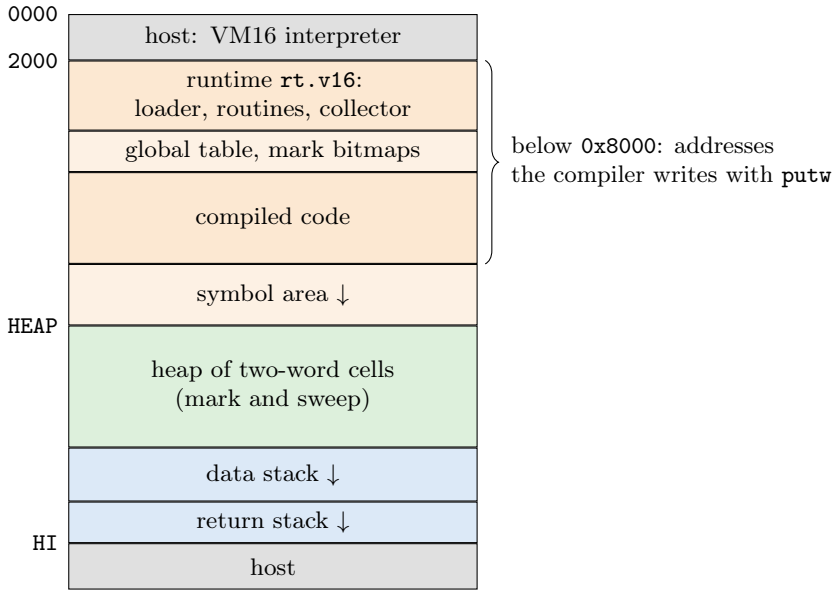


Figure 2: The planned memory map. `HEAP` is fixed when the runtime is assembled; with the heap from `0x8000` to `0xE000`, it holds 6144 cells.

An object file (`.mxo`) is a sequence of chunks, each a kind byte, a length word and that many bytes:

kind	meaning
`C`	code: copied to the next free address of the code area
`Y`	a symbol literal: interned, at the next address of the symbol area
`E`	the end; the body is the entry address, the initialisation code

The compiler writes each distinct symbol literal once, the first time it is used, and keeps the same running addresses as the loader, so it knows where every chunk lands.

The compiler reads the source once, with one token of lookahead, and has no syntax tree.

- The code of one function is collected in a list of bytes, newest first, and written as a `C` chunk when the function ends (section 7).
- An anonymous function is compiled into a buffer of its own, nested inside its parent's. When it ends, the compiler knows which outer locals it used, numbered in order of first use, and emits code in the parent to push their values, build the environment list and make the cell. A local used two levels down is captured by the function in between as well.
- A local that has been captured is marked, and so is a local that has been assigned; a local with both marks is an error.
- A call in `return`'s expression is compiled as a tail call when it is the last thing the expression does.
- The stores of every global function into its slot and the initialisers of the global variables, in program order, are collected and written at the end, followed by a call to `main` and a halt: the `E` chunk.

10 Bootstrapping

1. Write `rt.asm`, the runtime, in VM16 assembly, and `mxc.py`, a compiler from Mex to object files, in Python. It is the reference, and is written in the style of Mex (one pass, no syntax

tree, only the values of Mex), so that it can be translated by hand almost line by line.

2. Write `mxc.mx`, the same compiler in Mex, and test both on the same programs. Their output must be identical, byte for byte.
3. Compile `mxc.mx` with `mxc.py`, giving `mxc1.mxo`, and use that to compile `mxc.mx` again:

```
python3 mxc.py mxc.mx -o mxc1.mxo
cat mxc1.mxo mxc.mx | vm16 rt.v16 > mxc2.mxo
cmp mxc1.mxo mxc2.mxo
```

   The two files must be identical: the compiler has reached a fixpoint.
4. Repeat the last step on the Z80 build of VM16.

11 Open questions

- **Will it fit?** The heap is 24 KB: 6144 cells, less what the tables take. The compiler holds one function's code at a time, at a cell (4 bytes) per byte of code, its global table, and the symbols, which are compact. The first measurement should be the peak heap use of the compiler compiling itself, under `mxc.py` and `vm16 -c`.
- **Arrays.** A table with keys $0, 1, \ldots$ is an array, at about three words an element where a real array would need one. Boot's design gives a table a separate array part for these keys; it would cost a little more code in the runtime.
- **The capture rule** (a captured local is never assigned) keeps closures flat and locals on the stack, at the price of an occasional explicit box. Scheme's rule, shared mutable variables, would need environments on the heap.
- **15-bit integers** make addresses from `0x4000` up negative; the compiler should compare offsets, not addresses.
- **Strings as symbols.** Every symbol literal and every interned name stays in the symbol area for good. A compiler that builds many temporary names would fill it; one that interns only identifiers and messages should not.